HireLens-AI — Interview Preparation

Fresher Interview Q&A

HireLens-AI: an embedding-based resume ↔ job description matching tool with an interpretable skill-gap breakdown. Backend: FastAPI + sentence-transformers. Frontend: React + Vite. Deployed on Render (backend) and Vercel (frontend).

Project Overview

Q: Tell me about this project — what does it do and why did you build it?

HireLens-AI compares a resume against a job description and tells you two things — a match score based on how semantically similar they are, and a breakdown of which skills from the JD are missing from the resume. I built it because I wanted to go beyond basic keyword matching — most simple resume screeners just check for exact word overlap, but that misses cases where someone describes the same skill differently, like "built REST APIs" versus "developed backend services." So I used sentence embeddings to catch that kind of semantic similarity, but I kept a keyword-based skill list alongside it, because embeddings alone give you a score without telling you *why* — recruiters need to know specifically what's missing, not just a number.

Q: Walk me through what happens when a user uploads a resume, step by step.

First, the user uploads a PDF or text file and pastes in a job description. The file goes to a `/parse-resume` endpoint on my FastAPI backend, which extracts the raw text — I used `pdfplumber` for PDFs, going page by page and pulling the text out. Once I have both pieces of text — the resume and the JD — the frontend calls a second endpoint, `/match`. That runs two things: first, it embeds both texts into vectors using a sentence-transformers model and computes cosine similarity between them to get a 0 to 100 score. Second, it runs a keyword search against a fixed list of tech skills, and figures out which skills appear in both, which are in the JD but missing from the resume, and which are only in the resume. Both results come back together, and the frontend displays the score in a gauge along with the matched and missing skill lists.

Q: What tech stack did you use, and why those choices?

Backend is FastAPI with Python — I picked FastAPI because it's fast to build with, has built-in request validation through Pydantic, and auto-generates API docs, which made testing a lot easier during development. For the actual matching, I used `sentence-transformers` with a small pretrained model called MiniLM — it's lightweight enough to run on a free-tier server but still gives good semantic embeddings. Frontend is React with Vite, mainly because Vite gives a much faster dev experience than older tooling, and I wanted a simple component structure — an upload form, a results panel, and a score gauge. For deployment, I put the backend on Render and the frontend on Vercel, since both have generous free tiers and deploy directly from

GitHub.

Core Technical Concepts

Q: What is an embedding, in simple terms?

An embedding is a way of turning text into a list of numbers — a vector — such that texts with similar meaning end up close together in that number space, even if they don't share the same words. So "built REST APIs" and "developed backend web services" would produce vectors that are close to each other, because the model learned during training that those phrases mean similar things. In my project, I use a pretrained model called MiniLM to convert both the resume and the job description into these vectors.

Q: What's the difference between your semantic score and the skill-gap list? Why have both?

The semantic score is a single number — how similar the overall meaning of the resume and JD are. It's good at catching cases where someone describes their skills differently than the JD does, but it doesn't tell you specifically *what's* missing. The skill-gap list is the opposite — it's exact keyword matching against a fixed list of tech skills, so it's less flexible with wording, but it gives a precise, explainable breakdown: these skills matched, these are missing, these are extra. I used both because a score alone isn't actionable — if someone gets a 60%, they need to know *why*, and that's what the skill list gives them.

Q: What is cosine similarity measuring?

It measures how close two vectors are in meaning, based on the angle between them — not their length, just the direction they point in. It gives a value between 0 and 1: closer to 1 means the two texts are semantically very similar, closer to 0 means they're unrelated. So in my case, I convert the resume and job description into vectors, then use cosine similarity to see how aligned they are, and scale that into a 0 to 100 score.

Framework and Web Fundamentals

Q: What's FastAPI, and why did you pick it over Flask/Django?

FastAPI is a Python web framework for building APIs. I picked it over Flask because it has built-in request validation using Pydantic — so if a request is missing a field or has the wrong type, it automatically returns a clear error instead of me writing that validation manually. It also auto-generates interactive API documentation, which made it a lot easier to test my endpoints during development without needing a separate tool like Postman. Compared to Django, it's much lighter — Django comes with a lot of built-in stuff like an ORM and admin panel that I didn't need for a project this size.

Q: What's CORS, and why did you need to configure it?

CORS stands for Cross-Origin Resource Sharing — it's a browser security rule that blocks a webpage from calling an API hosted on a different domain, unless that API explicitly allows it. My frontend is deployed on Vercel and my backend is on Render, so they're on completely

different domains. Without configuring CORS, the browser would block every request from my frontend to my backend. I fixed it by adding my frontend's exact URL to an allowed-origins list in my FastAPI backend, which tells the browser "this specific domain is allowed to call me."

Q: What's a REST API? What HTTP methods does your app use and why?

A REST API is a way for a frontend and backend to communicate over HTTP, where each endpoint represents a resource or action, and you use different HTTP methods to indicate what kind of operation you're doing. My app uses two methods — GET for `/health`, which just checks if the server is alive and doesn't change anything, and POST for `/parse-resume` and `/match`, because both of those send data to the server — a file in one case, resume and JD text in the other — and the server processes that data and returns a result. I don't use PUT, PATCH, or DELETE anywhere because my app doesn't update or remove anything — every request is independent and stateless.

Q: Why is `/health` a GET and `/match` a POST?

GET is meant for requests that just retrieve information without changing anything or sending a meaningful payload — `/health` doesn't need any input, it just returns a status, so GET fits. `/match` needs to send actual data to the server — the resume text and job description — and GET requests aren't really meant to carry a body like that. POST is the standard choice whenever you're sending data for the server to process.

Q: What's the difference between synchronous and asynchronous code?

Synchronous code runs one operation at a time, blocking until each one finishes before moving to the next. Asynchronous code lets the program handle other work while waiting on something slow, like a network call or file read, instead of just sitting idle. In my app, `parse_resume` is `async def` because it does `await file.read()` — reading an uploaded file is I/O, so making it async means the server can handle other incoming requests while that read is happening. `match` isn't async, because it's not waiting on I/O — it's doing CPU work, running the embedding model and computing similarity. FastAPI actually handles that difference for you: a regular `def` endpoint automatically runs in a separate thread pool so it doesn't block the main event loop, while `async def` runs directly in that event loop and is expected to `await` things instead of blocking.

Deployment and Real Debugging Experience

Q: What was the hardest bug you ran into while building/deploying this?

When I deployed the backend to Render, it kept crashing with an out-of-memory error, even though my app itself is pretty lightweight. I dug into it and realized the issue wasn't my code — it was that installing `sentence-transformers` pulls in PyTorch as a dependency, and by default `pip` installs the full GPU-enabled build, which is hundreds of megabytes and loads a bunch of CUDA libraries into memory that are completely unused on a server with no GPU. Render's free tier only gives you 512MB, so it kept getting killed. The fix was forcing pip to install the CPU-only version of PyTorch instead, using PyTorch's separate CPU wheel index — that cut the memory footprint enough to fit under the limit. It taught me that a crash isn't always your application logic — sometimes it's a dependency pulling in more than you actually need.

Q: Did you deploy this? Where, and what problems came up?

Yes — backend on Render, frontend on Vercel, both connected to GitHub so they auto-deploy on push. Two real problems came up. First was that memory crash I just described. Second, once the backend was live, my frontend kept getting "failed to fetch" errors — turned out to be a CORS issue, since my frontend and backend are on different domains and I hadn't added my actual Vercel URL to the backend's allowed-origins list yet. I used the browser's Network tab to confirm it was specifically a CORS error and not something else, like a wrong environment variable, then updated the backend config and redeployed. Debugging that taught me to actually check the Network tab instead of guessing what's wrong from a generic error message.

Q: What would you do differently if you rebuilt it?

A few things. First, I'd add a Dockerfile from the start instead of relying on Render's default Python build — that would let me bake the model into the image at build time, so I'm not re-downloading it from Hugging Face on every fresh deploy, which currently makes the first request after a deploy pretty slow. Second, I'd make the skill matching a bit smarter — right now it's exact keyword matching, so it wouldn't catch something like "k8s" meaning the same thing as "kubernetes." I'd want to add some fuzzy or embedding-based matching there too. And I'd probably add basic logging or a small database, even just to track how many matches get run, since right now the app is completely stateless — nothing is saved.

Q: What are the limitations of your current approach?

A few I'm upfront about. One, the skill-gap detection is exact keyword matching against a fixed list, so it misses synonyms or abbreviations — "ML" versus "Machine Learning," for example. Two, it can't read scanned or image-based PDFs, since there's no real text layer for the PDF parser to extract — that would need OCR, which I haven't added. Three, the match-score thresholds — like what counts as a "strong" versus "weak" match — are just reasonable numbers I picked, not something tuned against real labeled data, so I can't claim they're scientifically calibrated. And since it's fully stateless with no database, there's no way to track history or compare multiple resumes over time right now.

Sanity Checks

Q: If someone pads their resume with keywords from the JD, would your system be fooled?

To some extent, yes, and I'd be upfront about that in an interview. The skill-gap list is purely keyword-based, so if someone just lists "Python, Kubernetes, AWS" with no real context, it would count as a match even without real experience behind it. The semantic score is a bit more resistant to this, because it's looking at overall meaning across the whole text rather than isolated keywords, but if someone pads a resume enough, it would likely still push the score up somewhat. This is a known weakness of keyword-based ATS-style systems in general, not just mine — a more robust version would need to weigh *context* around a skill, like years of experience or project descriptions, not just presence of the word.

Q: What data does your app store about the user?

Nothing, actually — the app is fully stateless. Looking at my schemas, a `/match` request just

takes `resume_text` and `job_description` as plain strings, the server computes a score and skill breakdown, and returns it immediately. Nothing gets written to a database or saved to disk anywhere — there isn't even a database in this project. Once the response is sent, the server has no memory of that request ever happening.